Countdown 上的后训练：SFT、IPO 与 RLOO

ShaneLiu04

https://github.com/ShaneLiu04

摘要

本文在 Countdown 算术任务上实现一条语言模型后训练管线：监督微调（SFT）、Identity Preference Optimization（IPO）与 Direct Preference Optimization（DPO），以及 leave-one-out REINFORCE（RLOO）。策略是 Qwen2.5-0.5B。奖励由可执行的等式检查给出：没有 `<answer>` 标签为 0，格式正确但等式错误为 0.1，用恰好给定的数字得到目标值为 1。基座模型在 50 道测试题上的 pass@1、pass@4、pass@8 都是 0.000。用搜索得到的短等式做一轮 SFT 之后，pass@1 升到 0.188（$n = 16$），留出负对数似然为 0.067。在 $\pi = \pi_{\text{ref}}$ 处，IPO 损失等于 $(1/(2\beta))^2$，DPO 损失等于 $\log 2$，实现与这两个恒等式一致。以序列对数概率之和为 Δ 时，IPO 在 $\beta = 0.1$ 上从 25.0 升到验证损失 31.07，偏好准确率停在 0.500。长度平均把 IPO 验证损失维持在 25.01。RLOO 的组内精确匹配率在四十步内围绕 0.156 波动。$K = 8$ 的两个种子在测试集上的 pass@1 分别是 0.253 和 0.223，pass@8 分别是 0.480 和 0.360。短程在线更新没有稳定地超过 SFT。全部数字来自 `metrics.jsonl` 与 pass@k 的 JSON 摘要。

关键词： SFT；IPO；DPO；RLOO；pass@k；Countdown；可验证奖励

1 问题

Countdown 给定一组整数和一个目标值，要求用每个整数恰好一次，以及 $+, -, \times, /$，写出一个等于目标值的等式。模型先在 `<think>` 中写下过程，再把等式放进 `<answer>`。这种任务有一个程序可以判定对错，因此奖励不依赖另一个打分模型 [Cobbe et al., 2021, Uesato et al., 2022, Lightman et al., 2023]。

基座模型不会发出作业要求的标签。没有标签时奖励恒为 0，一组样本的 leave-one-out 优势也是 0，策略梯度没有方向。SFT 的作用是先把输出分布移到奖励非零的区域，后面的偏好优化和在线强化学习才有比较对象 [Ouyang et al., 2022, Gandhi et al., 2025]。

本文实现作业中的三个训练阶段，并在同一套 50 道测试题上用无偏 pass@k 比较检查点 [Chen et al., 2021]。官方 starter code 保持不动。采样与参数更新放在同一个进程里，目标函数与 Ray worker 中的 `update` 相同。

2 方法

2.1 SFT

因果语言模型用位置 $t-1$ 的 logits 预测 token t。提示是左填充，回答是右填充。可训练位置必须同时是回答 token 且不是 padding，掩码再右移一位，与预测目标对齐。损失是这些位置上的平均负对数似然。提示 token 的梯度被掩掉，否则模型会把力气花在复读题目上。

2.2 IPO 与 DPO

记 $\Delta(y) = \log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)$。IPO 把 chosen 与 rejected 的对数比拉向一个常数 [Azar et al., 2024]：

$$L_{\text{IPO}} = \big(\Delta_w - \Delta_l - 1/(2\beta)\big)^2. \tag{1}$$

DPO 用同一个 Δ，但是 logistic 形式 [Rafailov et al., 2023]：

$$L_{\text{DPO}} = -\log \sigma\big(\beta(\Delta_w - \Delta_l)\big). \tag{2}$$

参考模型是同一份初始权重的第二次加载，参数 `requires_grad` 为假。Δ 可以是回答 token 的对数概率之和，也可以是长度平均。$\pi_\theta = \pi_{\text{ref}}$ 时 $\Delta = 0$，于是 IPO 损失是 $(1/(2\beta))^2$，DPO 损失是 $\log 2$。这两条恒等式用来核对实现，不依赖数据内容。

2.3 RLOO

对同一个提示采样 K 条回答，奖励为 $r_1, \ldots, r_K$。第 i 条的 leave-one-out 优势是 [Kool et al., 2019, Ahmadian et al., 2024]

$$\hat{A}_i = r_i - \frac{1}{K-1} \sum_{j \neq i} r_j. \tag{3}$$

组内奖励完全相同则优势为 0。在线采样使用 temperature 1、top-p 1，行为策略就是当前策略，重要性权重为 1。策略损失是 $-\mathbb{E}[\hat{A}_i \log \pi_\theta(y_i \mid x)]$，再加上 token

熵奖励和 $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$[Williams, 1992, Schulman et al., 2017]。熵和 KL 在词表维上按时间块计算，避免把整个 $[B, T, V]$ 转成 float32。

pass@k 使用无偏估计 $1 - \binom{n-c}{k} / \binom{n}{k}$，其中 n 是每个提示的采样数，c 是奖励为 1 的条数 [Chen et al., 2021]。$n < k$ 时该项不计算。

2.4 实现核对

单元测试有七项。掩码丢掉提示和 padding。负对数似然的反向传播不改动提示位置。参考策略上的 IPO 损失等于 $(1/(2\beta))^2$。大间隔时 DPO 损失接近 0。奖励 $[1, 0, 0, 0, 0.5, 0.5]$、组大小 2 的优势是 $[1, -1, 0, 0, 0, 0]$，组内奖励相等时优势为 0。序列对数概率默认是求和，打开平均开关后变成均值。均匀分布的熵为 $\log V$，两个相同分布的 KL 为 0。

3 实验设置

模型是 Qwen2.5-0.5B，词表大小约 151 936[Qwen Team, 2025]。硬件是一张 NVIDIA RTX PRO 6000 (97 887 MiB)，PyTorch 2.8.0+cu128。cgroup 的 CPU 配额是 22 核。注意力使用 scaled dot-product attention。`gradient_clipping`= 0 表示不裁剪。若把裁剪范数设成 0，`clip_grad_norm_` 会把梯度清零，因此只有阈值大于 0 才调用裁剪。

Countdown 训练集有 490 314 条，测试集有 50 条。测试集不参与任何训练。提示已经带有 Qwen 的 chat 模板，SFT 不再套第二次模板。SFT 的回答由穷举括号与运算得到：每个数字恰好用一次，等式的值等于目标。20 000 条均匀抽样全部有解，其中 19 000 条训练，1 000 条留出。回答是短句，形如 “`<think>` expr uses each given number once. `</think> <answer>` expr `</answer>`”。这教会格式和一条正确等式，不教会长的搜索过程。

SFT 使用 batch 64、梯度累积 2、学习率 5×10^{-5}、warmup 比例 0.05、余弦调度、裁剪范数 1、最大提示 512、最大回答 128，一个 epoch，296 次优化器更新。偏好数据有 48 311 对训练样本和 138 对验证样本，列包括 `query`、`response_ws`、`response_ls`。`query` 还不是 chat 模板，因此 IPO 的数据加载会套模板。求和版本的 IPO ($\beta = 0.1$) 训练 120 步，DPO ($\beta = 0.1$) 和 IPO ($\beta = 0.5$) 各 80 步。长度平均版本各 250 步。学习率 5×10^{-6}，batch 16，累积 2，提示上限 384，回答上限 256。调度长度按 `max_steps` 计算，避免短跑仍处在按全数据估计的 warmup 里。

RLOO 从 SFT 检查点出发，在训练集的前 2 000 条上循环。默认每步 4 个提示、组大小 8、最多新生成 128 个 token、学习率 10^{-5}、熵系数 0.001、KL 系数 0.001、四十步。对照包括组大小 2（每步 8 个提示，批大小仍是 16）、去掉熵和 KL、第二个随机种子，以及修正停止符之后的四十步和一百二十步。评估温度 0.6、top-p 0.95。SFT 的 pass@k 用每个提示 16 条回答，其余检查点用 8 条。估计量对同一 k 无偏，方差不同。

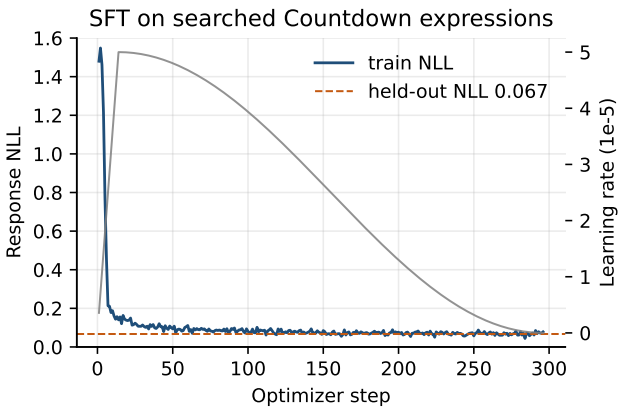


图 1: 搜索等式上的回答负对数似然。灰线是学习率，单位 10^{-5}，用右轴。橙色虚线是 1 000 条留出样本的负对数似然。

官方脚本用 vLLM 采样、用 Ray 更新。本实验改成进程内的 Hugging Face `generate`。原因是 Blackwell 上的 vLLM 轮子体积大且不一定匹配当前 CUDA。目标函数、on-policy 的 temperature 和 top-p、以及优势定义与课程脚本一致。这个替换改变的是吞吐和停止符的实现，不是损失的数学形式。

4 SFT

图 1 里训练损失从 1.480 降到 0.078，留出损失是 0.067。下降主要发生在前二十步，之后损失贴着留出值波动，学习率按余弦降到 0。留出集与训练集来自同一种搜索答案，所以低负对数似然说明模型记住了这种短格式，不能单独说明它会解没见过的数字组合。

基座的 pass@1 是 0.000。SFT 之后 pass@1 为 0.188，pass@4 为 0.388，pass@8 为 0.475，pass@16 为 0.540。抽样里可以看到正确的标签和一条用全给定数字的等式，但等式的值经常不等于目标。格式奖励因此接近饱和，精确奖励仍然稀疏。这正是后面 RLOO 能算出非零优势的原因。

5 偏好优化

图 2 的第一步损失与恒等式一致：$\beta = 0.1$ 的 IPO 从 25.0 起步，$\beta = 0.5$ 从 1 起步，DPO 从 0.693 起步。求和版本的 IPO 随后剧烈振荡，验证损失升到 31.07，

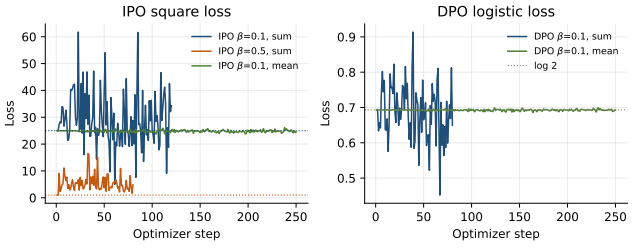


图 2: 左：IPO。点线是 $\pi = \pi_{\text{ref}}$ 时的恒等式，$\beta = 0.1$ 为 25，$\beta = 0.5$ 为 1。右：DPO，点线是 $\log 2$。sum 表示 Δ 用序列对数概率之和，mean 表示长度平均。

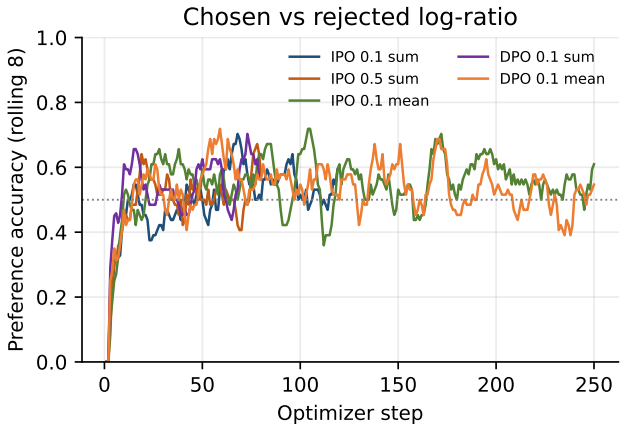


图 3: 滚动八步的偏好准确率：$\Delta_w > \Delta_l$ 的比例。虚线是 0.5。

验证准确率 0.500。$\beta = 0.5$ 的验证损失是 5.97，准确率 0.493。长度平均的 IPO 验证损失是 25.01，准确率 0.507，曲线贴着 25 这条恒等线。DPO 求和版本的训练损失降到 0.650，验证损失是 0.699，验证准确率 0.566。长度平均的 DPO 验证损失是 0.694，准确率 0.507。

平方损失对 Δ 的尺度敏感。回答有一两百个 token 时，对数概率之和的微小相对变化会让 $\Delta_w - \Delta_l$ 离开 $1/(2\beta) = 5$，平方项就到几十。长度平均把目标间隔放在每个 token 上，更新更稳，但在这 250 步里也没有把准确率拉离 0.5。图 3 的五条曲线都在 0.5 附近。偏好数据里的 chosen 与 rejected 都是带 `<think>` 的长文本，而 SFT 只见过一行等式。参考策略给两条长回答的对数概率都很低，短程更新分不开它们。

6 RLOO

图 5 里灰色的“没有标签”几乎看不见，橙色的格式奖励占绝大多数，绿色的精确匹配大约在 0.1 到 0.3 之间。四十步上精确匹配率的平均值是 0.156，奖励平均值是 0.241。组内既有 1 又有 0.1 时优势非零。全体都是 0.1 或全体都是 1 时，日志里的策略损失就是 0，这与公式一致，也说明这一步没有学习信号。

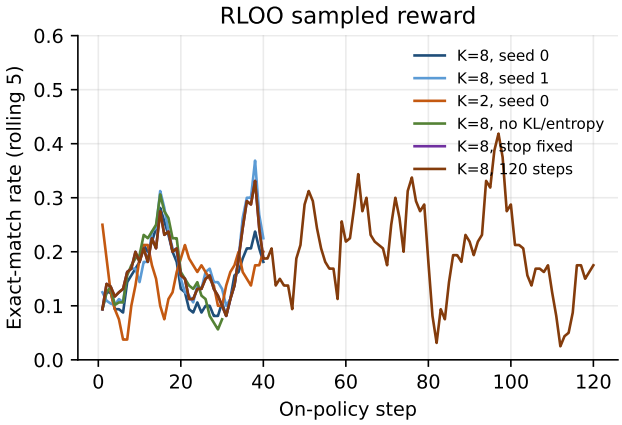


图 4: 在线采样的精确匹配率，五点滚动平均。K 是组大小。stop fixed 使用能匹配前导空格的 `</answer>` token 序列。

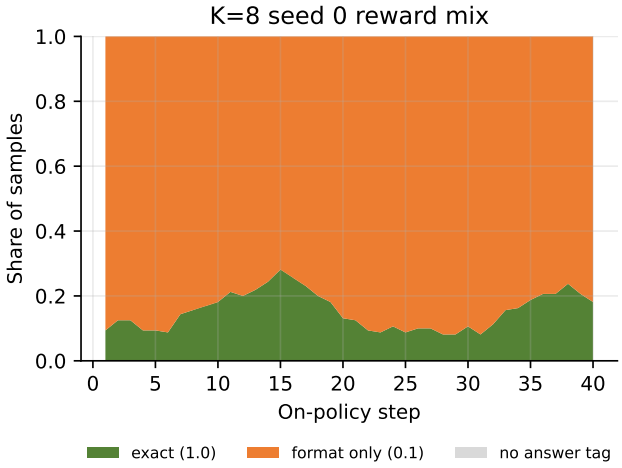


图 5: $K = 8$、种子 0、未修正停止符时的奖励构成，五点滚动平均。SFT 之后几乎每条样本都有答案标签，绿色部分才是等式正确的比例。

图 4 的四条短跑都在 0.05 到 0.35 之间来回，没有单调上升。每步只有 4 个提示、32 条样本，滚动平均仍然很吵。$K = 2$ 的基线只有另一条样本，方差更大，测试集上的 pass@8 也更低。去掉 KL 和熵之后，在线精确率没有分开，测试集 pass@8 略高，见下一节。

第一批 RLOO 把停止序列设成 `encode("</answer>")`，得到 `</`、`answer`、`>`。模型实际生成的是带前导空格的 `Ġ</`、`answer`、`>`。尾部对不上，生成会越过答案标签，继续吐出 `</system>` 和 `</user>` 的文本。这些 token 仍在回答掩码里，序列级优势会加到它们上面。修正后的停止同时接受两种切分，并在第一个闭合标签处截断损失。四十步修正运行的在线精确率均值是 0.175。它没有高于未修正的运行。把同一设置延长到一百二十步后，测试 pass@1 是 0.245，pass@8 是 0.340，后者低于 SFT 的 0.475。停止符决定优势落在哪些 token 上。在这个批大小下，加

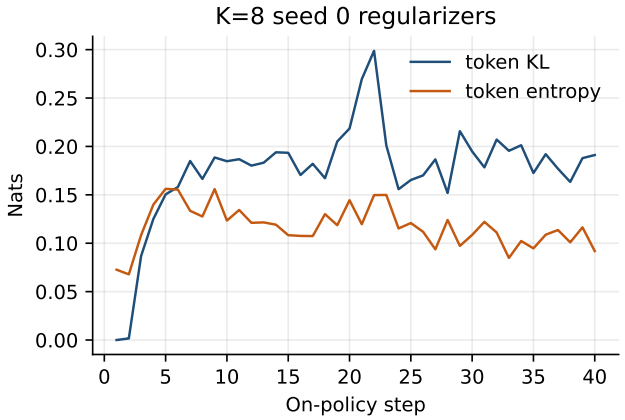


图 6: 同一条 $K = 8$ 运行的 token KL 与 token 熵。系数都是 0.001。

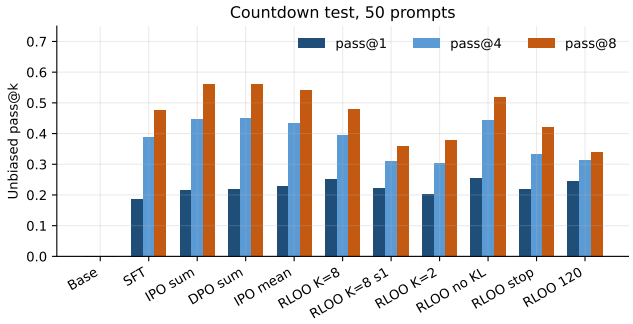


图 7: 测试集 50 题的无偏 pass@k。SFT 每个提示采样 16 条，其余为 8 条。Base 三项都是 0，柱高与坐标轴重合。

长步数并没有把 pass@8 拉过 SFT。

7 pass@k

表 1 和图 7 放在一起读。SFT 把 pass@1 从 0.000 抬到 0.188。求和 IPO、求和 DPO 和长度平均 IPO 的 pass@1 分别是 0.215、0.220 和 0.230，pass@8 分别是 0.560、0.560 和 0.540。它们高于 SFT 的 pass@1，但 n 和种子都只有一份，差距在几个百分点，和偏好准确率仍接近 0.5 是同一件事的两面：检查点还没有稳定地偏好 chosen。

$K = 8$ 的 RLOO 在种子 0 上 pass@1 为 0.253，种子 1 为 0.223。pass@8 从 0.480 降到 0.360。$K = 2$ 的 pass@8 是 0.380。去掉 KL 与熵的 pass@1 是 0.255，pass@8 是 0.520，是这组 RLOO 里 pass@8 最高的一条，仍然只有一个种子。修正停止符的四十步 pass@1 是 0.217，低于未修正的种子 0。

pass@1 上升而 pass@8 不上升，意味着单次采样更常碰对，八次采样里“至少一次正确”的概率没有一起变好。四十到一百二十步、每步四个提示，不足以把这种差别从采样噪声里分开。官方设置是 batch 128、组

表 1: 测试集 pass@k。SFT 的 $n = 16$，其余 $n = 8$。

	pass@1	pass@4	pass@8
Base	0.000	0.000	0.000
SFT	0.188	0.388	0.475
IPO $\beta = 0.1$，sum	0.215	0.447	0.560
DPO $\beta = 0.1$，sum	0.220	0.449	0.560
IPO $\beta = 0.1$，mean	0.230	0.434	0.540
RLOO $K = 8$，seed 0	0.253	0.395	0.480
RLOO $K = 8$，seed 1	0.223	0.311	0.360
RLOO $K = 2$	0.203	0.303	0.380
RLOO $K = 8$，无 KL	0.255	0.442	0.520
RLOO $K = 8$，修正停止	0.217	0.333	0.420
RLOO $K = 8$，120 步	0.245	0.313	0.340

大小 8、累积 128、100 步。这里的在线批远小于那个设置。结论只覆盖这个预算：SFT 提供格式，短程 IPO 与 RLOO 没有给出稳定的 pass@8 增益。

8 扩展：长 think、优势归一化与验证器重排

上一节的在线批是每步 4 个提示、最多 128 个新 token，SFT 目标是一行等式。扩展换成数据集里的长 `<think>`，并用 vLLM 做 on-policy 采样。策略仍是 Qwen2.5-0.5B。留出集 500 题、测试集 50 题，每题 16 条，温度 0.6、top-p 0.95。在线训练每步 32 个提示、组大小 8、100 步，学习率 10^{-5}，熵系数和 KL 系数都是 0.001。三种优势共用同一条序列对数概率之和，差别只在组内统计：

$$\hat{A}_i^{\mathrm{RLOO}} = r_i - \frac{1}{K-1}\sum_{j \neq i} r_j, \tag{4}$$

$$\hat{A}_i^{\mathrm{GRPO}} = (r_i - \bar{r})/(\sigma + \epsilon), \tag{5}$$

$$\hat{A}_i^{\mathrm{Dr}} = r_i - \bar{r}. \tag{6}$$

Dr. GRPO 去掉的是标准差缩放 [Shao et al., 2024]。本实现没有再把损失按长度平均，所以它不是“除以回答长度”的那一项消融。课程学习把前一半提示换成三数字题，后一半换成四数字题。验证器重排发生在采样之后：在 k 条回答里用 Countdown 检查器挑选，对照是均匀抽取、多数投票、最大平均对数概率，以及只检查数字是否用全。

长 think SFT 的留出 pass@1 是 0.272，pass@16 是 0.662。短等式 SFT 是 0.197 和 0.554。从长 think 出发，RLOO 三个种子的 pass@1 是 0.525 ± 0.004，pass@16 是 0.676 ± 0.029。GRPO 是 0.512 ± 0.016 / 0.648 ± 0.016，Dr. GRPO 是 0.534 ± 0.008 / 0.675 ± 0.019，课程学习是 0.483 ± 0.037 / 0.651 ± 0.013，从官方 SFT 检查点出

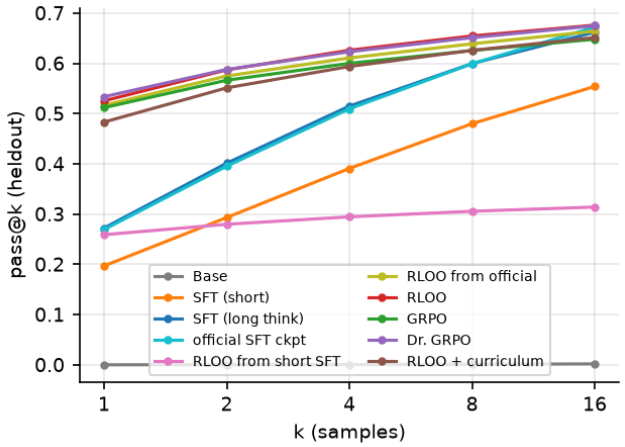


图 8: 留出集 500 题的 pass@k。长 think SFT 把 pass@16 抬到 0.662。其后的在线方法把 pass@1 抬到 0.52 附近，pass@16 仍停在同一量级。短等式上的 RLOO 把 pass@16 压到 0.314。

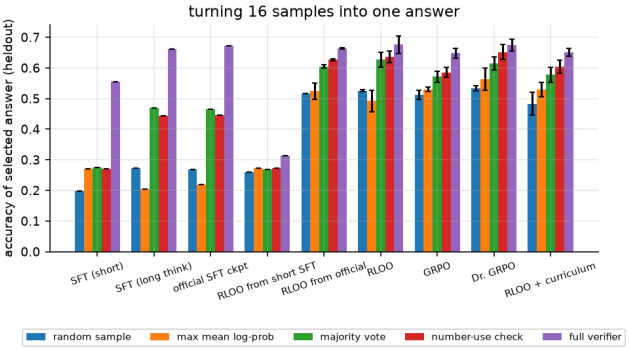


图 9: 留出集上、预算 $k = 16$ 时的选择器。全验证器与 pass@16 重合，因为它就是奖励函数。最大平均对数概率低于均匀抽取。

发的 RLOO 是 0.517 ± 0.001 / 0.664 ± 0.002。宏里的 $\pm$ 是种子间标准差，种子数写在 `extension_macros.tex`。短等式上的 RLOO 留出 pass@1 只有 0.259，pass@16 掉到 0.314，低于它自己的 SFT。在线更新可以收紧分布，也可以把覆盖打掉。

pass@1 从大约 0.27 升到 0.52–0.54，而 pass@16 仍在 0.64–0.68。这是分布变尖，不是解题覆盖变宽。GRPO 与 Dr. GRPO 的差距小于种子标准差能分开的范围时，不能把“去掉标准差”说成稳定增益。课程学习没有超过同一预算的普通 RLOO。

图 9 里的全验证器使用题目给出的目标和数字，不需要额外标注。它在 k 条里挑出正确等式，所以 $k = 16$ 时等于 pass@16。最大平均对数概率会偏向短而流利的错答案。多数投票和数字检查介于二者之间。测试时多采样再检查，收回的是 pass@16 里已经存在的解，不是训练没覆盖到的解。

9 局限

SFT 目标是搜索器写出的一行等式，不是数据集里那种多步 `<think>`。模型可以在标签内放进一条等式，却没有被要求展示搜索。偏好对来自长回答，和 SFT 的短回答不在同一个长度上，求和 IPO 因此不稳。

RLOO 只跑了一百二十步量级，组内提示数是 4 而不是 128。两个种子的 pass@8 已经相差 0.480 与 0.360。更多种子会收紧区间，但不会把这个预算变成官方的全量训练。评估温度 0.6 与训练温度 1 不同，pass@k 衡量的是评估分布，不是训练时的在线奖励。

进程内生成没有使用 vLLM 的连续批处理。停止符的 token 切分必须和模型真正生成的字节一致，否则序列级优势会记到答案标签之后的 token 上。这一点已在修正运行里单独列出。

参考文献

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Rémi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective STaRs. *arXiv preprint arXiv:2503.01307*, 2025.

Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 REINFORCE samples, get a baseline for free! In *ICLR Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. In *arXiv preprint arXiv:2211.14275*, 2022.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.